Manual de Soporte a Operaciones de Framework 4 Datahub para Banco Internacional del Perú-Interbank

Datos de control

Autores:	Jhonny Quispe, Gabriela Motta, Andre Ramos, Martin Bautista
Dueño:	
Versión:	1.0
Fecha:	Marzo 2026
Ubicación:	Lima, Peru

Control de versiones

NUMERO	FECHA	ACTUALIADO POR	CAMBIOS REALIZADOS
1.0	08-Mar-2026		Borrador inicial

Lista de Distribución

NOMBRE	EMAIL	COMPAÑIA
Chumioque Pachas, Antonio Gustavo	achumioque@intercorp.com.pe	IBK

Tabla de contenido

Introducción 4
Arquitectura 5
Arquitectura de la solución (actual) 5
Arquitectura de la solución (propuesta) 6
Framework v4 7
Ecosistema Framework v4 8
Componentes y versiones 8
Documentación para los Data Engineer que van a 9
configurar un proceso 9
Configuración ON-PREMISE 9
Ingesta de proceso individual 10
Configuración ON-CLOUD 15
Config process agrupado 20
Cierre de Documento 23

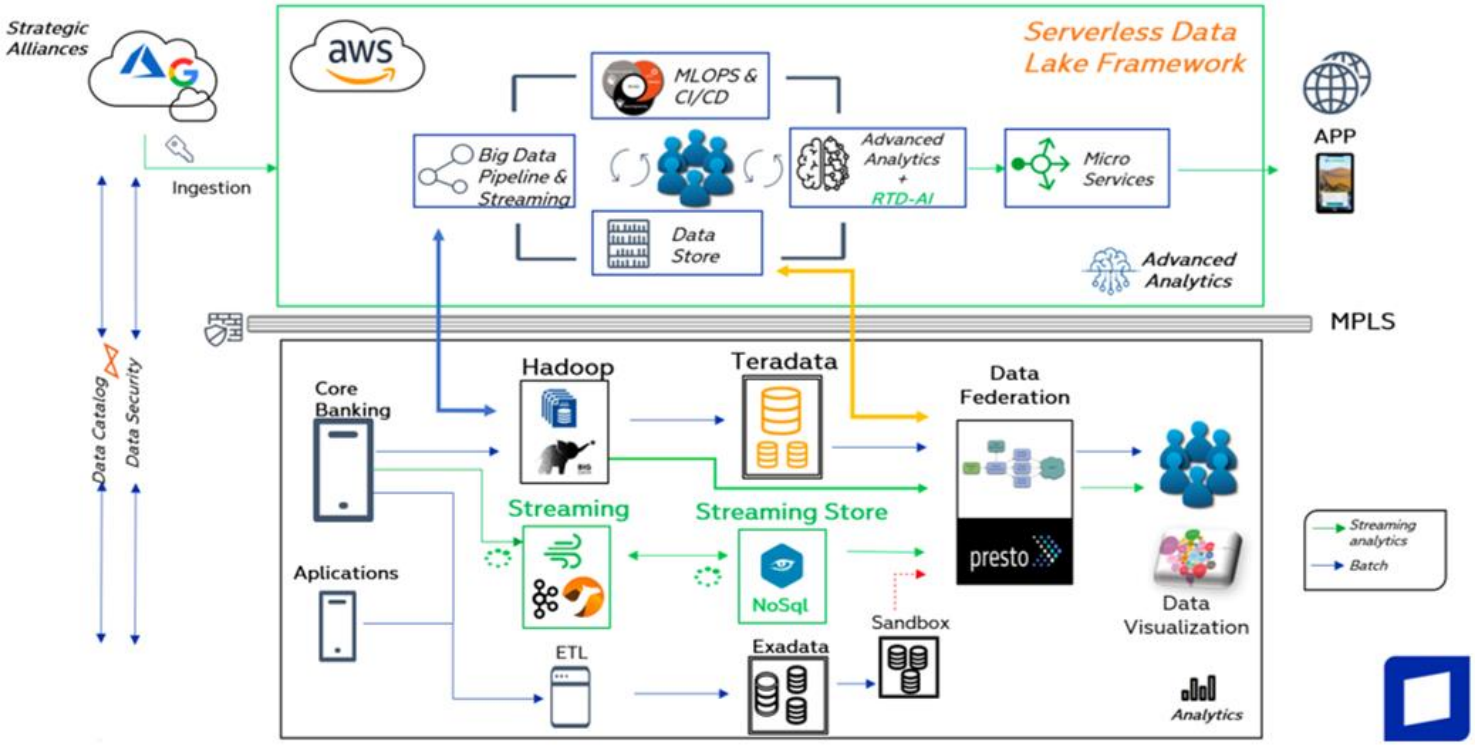
Introducción

El manual de operaciones del servicio **Framework 4** (Run Book) constituye una guía de referencia para **INTERBANK** y para el personal de **Servicios Administrados** responsable de este framework. Este documento, elaborado de forma conjunta por **DWCONSULWARE** e **INTERBANK**, describe los procesos, procedimientos y lineamientos técnicos acordados para asegurar la operación, administración y soporte del ecosistema de datos del cliente.

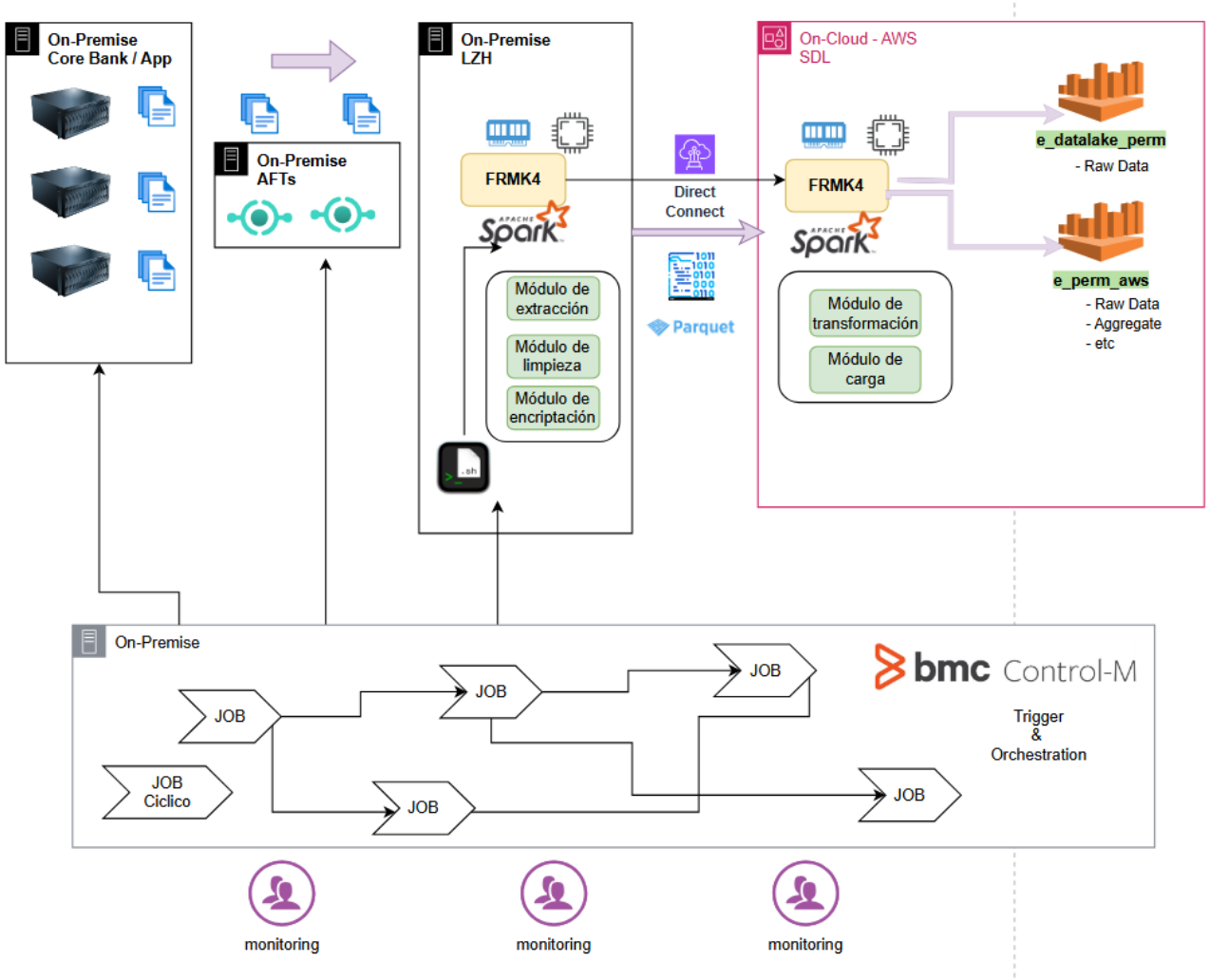
El contenido se enfoca en las actividades de configuración y continuidad operativa del **Framework 4**, con el objetivo de brindar una gestión eficiente, estandarizada y trazable en el entorno de integración de datos del **Datahub**. Asimismo, establece las bases para una atención oportuna y una mejora continua del servicio, alineada con las necesidades operativas del negocio.

Arquitectura

Arquitectura de la solución (actual)



Arquitectura de la solución (propuesta)



Framework v4

Es una solución modular de procesamiento de datos, implementada principalmente en **Spark (Python)** y orientada a la ejecución de flujos de ingesta, validación, transformación, seguridad y publicación dentro del ecosistema **Datahub**. A diferencia del enfoque monolítico de versiones anteriores, Framework v4 organiza su operación en componentes desacoplados y configurables, lo que facilita la mantenibilidad, escalabilidad y evolución funcional del servicio.

Framework v4 concentra la lógica necesaria para soportar las ingestas del entorno actual, incorporando capacidades de orquestación por proceso, uso de archivos de configuración, trazabilidad operativa y control de errores, permitiendo una gestión más robusta del ciclo de vida de los datos.

En capítulos posteriores se detallará la arquitectura y el funcionamiento de cada módulo que conforma Framework v4, así como los lineamientos técnicos para su administración y soporte operativo continuo.

Ecosistema Framework v4

Componentes y versiones

El **Framework 4** utiliza como base **Python 3** para la ejecución de sus módulos y **Apache Spark** para el procesamiento distribuido de datos.
A nivel de motor de procesamiento, se trabaja con distribución local de Spark en la ruta del framework, por lo que el servidor debe contar con prerrequisitos de sistema y Java compatibles antes de la instalación.

Para la configuración de entorno operativo (PRD/UAT/DES), se mantiene el esquema de usuarios por ambiente y archivos de inicialización del entorno:

Ambiente	Usuario
PRD	prodAdmin
UAT	certAdmin
DES	desaAdmin

Nombre | Ruta
IncludeConfig.sh | /home/<ambiente>
envDH.ini | /home/<ambiente>/bin

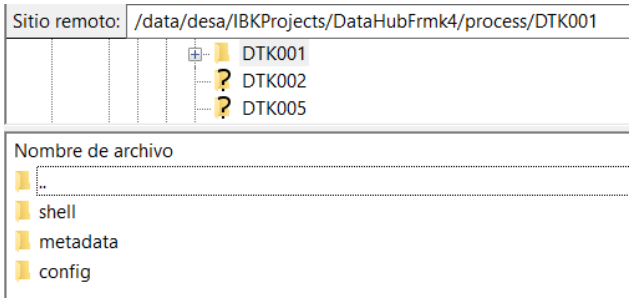
A continuación, se detallan los componentes principales del Framework 4 (referencia técnica de la implementación actual):

Componente	Versión
Python	3.6
Apache Spark	3.5.8 (spark-3.5.8-bin-hadoop3)
PySpark	3.5.8
Py4J	0.10.9.7
Java/JVM	Version compatible con Spark 3.5x

Documentación para los Data Engineer que van a configurar un proceso

Para realizar una configuración end-to-end de un proceso de ingesta, la implementación debe completarse tanto en **on-premise** como en **on-cloud**.
En **on-premise** se configura la lógica técnica de entrada: definición de columnas (metadata/input_columns.txt), reglas de limpieza/preproceso (shell/*.sh) y parametrización principal del proceso (config/config_process.json: rutas, formato, delimitador, clean, reject, etc.).
En **on-cloud** se configura un item en dynamo, el cual se parametriza la transformación y los targets de carga como Athena y S3.

Configuración ON-PREMISE
En el servidor de on-premise se debe crear una estructura de carpetas similar a la siguiente imagen



shell: contiene scripts ejecutables de soporte del proceso (normalmente limpieza/preprocesamiento del archivo antes del extract).

metadata: contiene definiciones de metadatos del archivo de entrada; por ej emplo, input_columns.txt define el nombre y orden de las columnas que leerá el framework.

config: contiene la configuracion del proceso (principalmente config_process.json): tipo de extraccion, rutas de input, delimitador/header, archivo de columnas ("columns": "input_columns.txt"), script de limpieza (clean.process) y flags de control (reject, data_protection, etc.).

shell/limpia_*.sh transforma/normaliza el input antes del reject/carga.

El framework genera metadata.json en output_final como evidencia técnica del resultado.

Ingesta de proceso individual

En el proceso **individual**, cada ejecución trabaja con **una sola fuente** (un archivo, un proveedor o una interfaz) de forma aislada.
Esto permite que cada fuente tenga su propio flujo: validación, trazabilidad, error handling y reintentos independientes.
La ventaja principal es el **aislamiento**: si una fuente falla, no detiene las demás; además, el diagnóstico es más simple porque todo el resultado corresponde a una sola entrada.
Es ideal cuando las fuentes tienen comportamientos distintos, ventanas de llegada diferentes o requisitos particulares de negocio.

En las ingesta de proceso individual existen dos tipos CM y CC.

Ejemplo de json de configuración :

```
{
  "process_id": "DTK001",
  "process_type": "CM",
  "process_type_group_cc":"",
  "process_modules": [
    {
      "extract": [
        {
          "extract_type": "file",
          "properties": {
            "format": "text",
            "path": "/data/desa/landingzone/hadoop/interfacesUser/input",
            "source_name": "dataset_abc",
            "extension": "txt",
            "delimiter": "|",
            "header": "n",
            "columns": "input_columns.txt",
            "columns_key": "CODUNIC,NRODOC",
            "columns_critical": "CODUNIC,FECAPERT,SALDO",
            "original_enabled_ind": "Y",
            "clean": {
              "type": "shell_script",
              "process": "limpia_NGSH.sh"
            },
            "reject": {
              "threshold_percent": "0",
              "valid_number_columns" : "Y",
              "valid_characters" : "Y"
            },
            "data_protection": {
                "columns_protection": [
                  {"column_name": "NRODOC" , "protection_name" : "MASKING_HASH_SHA256_VOLT" }
                 ,{"column_name": "TELEF"  , "protection_name" : "MASKING_HASH_SHA256_VOLT" }
                ]
            }
          }
        }
      ]
    }
  ]
}
```

DTK001.json

Donde:

Propiedad	Valor	Descripción
process_id	string	Identificador único del proceso. Debe coincidir con la carpeta del proceso (process/<process_id>). Se usa para rutas, logs, metadata y salida final.
process_type	string	Tipo de proceso (CM, CC). Se usa para filtrado de ejecución en escaneo de procesos.
process_type_group_cc	string	Grupo para ejecuciones cíclicas de tipo CC. Si se invoca cc_process, se filtra procesos por este valor.
process_modules	array	Lista de módulos. Actualmente el flujo usa módulo extract. (clean, reject, data_protection)

Modulo extract

Propiedad	Descripcion	Valores
extract_type	indica el tipo de extraccion	file

Properties {}				
Propiedad	**Tipo**	**Requerido**	**Comportamiento**	**Ejemplo (DTK001)**
format	string	Sí	Formato esperado del input (text, etc.).	text
path	string	Sí	Ruta landing de entrada donde se deposita el archivo.	/data/desa/landingzone/hadoop/interfacesUser/input
source_name	string	Sí	Nombre lógico del archivo fuente; se busca como source_name.<ext> o source_name_<fecha>.<ext>.	dataset_abc
extension	string	Sí	Extensión del archivo de entrada.	txt

delimiter	string	Sí	Delimitador para lectura/split y reject.	|
header	string	Sí	Indicador de cabecera (Y/N).	n
columns	string	Sí	Archivo de metadata de columnas (process/<process_id>/metadata/<columns>).	input_columns.txt
columns_key	string	Opcional	Referencial para calidad/negocio (llaves).	CODUNIC,NRODOC
columns_critical	string	Opcional	Referencial para columnas críticas de calidad/negocio.	CODUNIC,FECAPERT,SALDO
original_enabled_ind	string	Opcional	Y: hace backup del original; N: no hace backup (default: Y).	Y

Modulo Clean

```
"clean": {
  "type": "shell_script",
  "process": "limpia_NGSH.sh"
},
```

Modulo Clean			
Propiedad	**Descripcion**	**Valores**	**Descripcion**
type	Describe el tipo de limpieza	shell_script	son shell que recibe parametros de entrada y procesa la limpieza, ver ejemplo 0001.sh
		linux_cmd	son comandos que en tiempo de ejecucion son ejecutados linea a linea, ver ejemplo 0002.sh

EJEMPLO SHELL_SCRIPT:

limpia_NGSH.sh

EJEMPLO LINUX_CMD:

Limpia_CYBER_BLANCO_BPE.sh

Modulo reject:

```
"reject": {
  "threshold_percent": "0",
  "valid_number_columns" : "Y",
  "valid_characters" : "Y"
},
```

Modulo Reject				
Propiedad	**Tipo**	**Requerido**	**Comportamiento**	**Valores**
enabled_ind	string	Opcional	Si falta, reject queda habilitado. Si N, no aplica reject.	Y = habilita, N = deshabilita. Si se informa y no es Y, se toma como deshabilitado.
threshold_percent	string/number	Sí recomendado	Umbral máximo de inválidos (%). Si se supera, falla el proceso.	Numérico (0 default). Recomendado 0 a 100. Ej: 0, 15, 2.5, "15".
engine_process_reject	string	Opcional	Motor de reject.	shell-script (default) o spark. También acepta shell_script (se normaliza).
valid_number_columns	string	Opcional	Valida cantidad de columnas por fila.	Y/N. Default: Y.
valid_characters	string	Opcional	Valida caracteres inválidos/no imprimibles.	Y/N. Default: Y.

Reglas importantes:

- Si reject no existe, el framework aplica reject con tolerancia 0%.
- En modo shell, genera *_registros_validos.* y *_invalidos.*.
- Si umbral es excedido, termina con error.

Modulo data_protection

```
"data_protection": {
    "columns_protection": [
       {"column_name": "NRODOC" , "protection_name" : "MASKING_HASH_SHA256_VOLT" }
      ,{"column_name": "TELEF"  , "protection_name" : "MASKING_HASH_SHA256_VOLT" }
    ]
}
```

Modulo Data Protection				
Propiedad	**Tipo**	**Requerido**	**Comportamiento**	**Valores**
enabled_ind	string	Opcional	Si falta, default efectivo es habilitado cuando hay columnas configuradas.	Y, N, o omitido. Omitido equivale a Y, pero solo se aplica si columns_protection existe y tiene al menos 1 elemento.
columns_protection	array	Sí para aplicar	Lista de columnas y tipo de protección.	[] o lista de objetos: [{"column_name":"<column>","protection_name":"<tipo>"}]. Ejemplo: [{"column_name":"NRODOC","protection_name":"MASKING_HASH_SHA256_VOLT"}].
column_name	string	Sí	Columna destino (match case-insensitive).	Nombre de columna existente en el dataset/metadata. Ejemplos: NRODOC, TELEF. El match es case-insensitive.
protection_name	string	Sí	Algoritmo de protección (MASKING_HASH_SHA256_VOLT/ENCRYPTION_AES_GCM).	MASKING_HASH_SHA256_VOLT o ENCRYPTION_AES_GCM (acepta mayúsc/minúsc, se normaliza). Cualquier otro valor falla con error de tipo no soportado.

Notas:

- Si columns_protection está vacío, no aplica protección.
- Si una columna no existe, se registra en log.

Configuración ON-CLOUD

En el flujo OnCloud, el DE debe configurar una tabla en DynamoDB, la cual funcionará como componente de parametrización y control del proceso. A través de esta configuración, se podrán definir los metadatos, reglas de conexión y parámetros necesarios para habilitar la carga hacia distintas fuentes o destinos de información, como Amazon S3 y Amazon Athena.

En la parte cloud se debe configurar tres servicios

a) Config: dynamo DB - tabla: aw-dev-ddb-sdl-datahub-config-process-01
b) Archivo SQL en ruta s3 – ruta sql (Opcional): s3://……

A continuación se detallara cada parte:

1.1 Configurar de Json en dynamo

El DE debe configurar sus procesos en la tabla de dynamo: aw-dev-ddb-sdl-datahub-config-process-01

```
{
 "process_id": "DTK4001",
 "process_modules": [
  {
   "targets": [
    {
     "load_type": "athena",
     "properties": {
      "format": "parquet",
      "schema": "d_datalake_perm",
      "table": "dtk4001_frmk4"
     }
    },
    {
     "load_type": "athena",
     "properties": {
      "format": "parquet",
      "schema": "d_perm_aws",
      "table": "dtk4001_frmk4"
     },
     "transform_id": "t1"
    }
   ],
   "transform": [
    {
     "transform_id": "t1",
     "transform_sql": "query_DTK4001.sql"
    }
   ]
  }
 ]
}
```

aw-dev-ddb-sdl-datahub-config-process-01_DTK001.json

Este ítem define la configuración de proceso para FRMK4 y se guarda en la tabla de procesos (aw-dev-ddb-sdl-datahub-config-process-01) con clave primaria process_id = DTK4001. Su función es indicar qué transformaciones ejecutar y a qué destinos cargar los resultados.

¿Qué hace cada bloque?

process_id identifica el proceso que recibe el job (--process_id DTK4001).

transform define las transformaciones disponibles. Aquí solo existe t1, que ejecuta el SQL query_DTK4001.sql.

targets define salidas del proceso:

- El target S3 no tiene transform_id, por lo que carga el dataset base del flujo (sin forzar una transformación específica).
- Los dos targets athena usan transform_id = t1, por lo que ambos publican el resultado transformado t1 en dos esquemas distintos.

¿Cómo participa en la ejecución?

Cuando corre el job de EMR:

1. Se carga este ítem desde Dynamo por process_id.
2. El framework identifica que debe ejecutar la transformación t1.
3. Ejecuta query_DTK4001.sql.
4. Publica resultados a:
    - S3: s3://aw-dev-s3-sdl-datahub-process-058528764918-01/tmp/load_s3_dtk4001 en parquet.
    - Athena: d_datalake_perm.dtk4001_frmk4 en parquet.
    - Athena: d_perm_aws.dtk4001_frmk4 en parquet.

Requisitos para que funcione

Debe existir el archivo query_DTK4001.sql en la ruta de SQL que usa el framework (configurada en global config).
El rol de ejecución de EMR debe tener permisos de escritura en el bucket S3 y en el catálogo/tablas de Athena/Glue.
Los esquemas de Athena deben existir o estar habilitados para creación según el loader.

Parámetro	Descripción	Valor a configurar (ejemplo)	Adicional configuración
process_id	Identificador único del proceso en Dynamo. Debe ser estable y no repetirse. (debe ser el mismo id que se configuro en el path config de onpremise)	DTK4001	NA
process_modules	Lista de módulos del proceso. Normalmente usas 1 módulo con transform y targets.	[{ ... }]	NA
process_modules[0].transform	Lista de transformaciones SQL disponibles para este proceso.	[{ "transform_id": "t1", "transform_sql":	NA

		"query_DTK4001.sql" }]	
process_modules[0].transform[0].transform_id	ID interno de la transformación; luego se referencia desde los targets.	t1	NA
process_modules[0].transform[0].transform_sql	Nombre del archivo SQL que ejecuta la transformación. Debe existir en la ruta SQL del framework.	query_DTK4001.sql	El DE debe subir un dml en sql spark al sgte bucket: s3://aw-dev-s3-sdl-datahub-process-058528764918-01/process/DTK4001/sql/query_DTK4001.sql
process_modules[0].targets	Lista de salidas/cargas del proceso. Puedes tener varias.	S3 + Athena + Athena	NA
process_modules[0].targets[n].load_type	Tipo de carga para cada target.	S3 o athena	En caso sea Athena, el DE debe crear el DDL de la tabla en athena.
process_modules[0].targets[n].transform_id	Indica qué transformación usar para ese target. Si se omite, usa dataset base.	t1 (en targets Athena), omitido en target S3	NA
process_modules[0].targets[n].properties.format	Formato de salida del target.	parquet	NA
process_modules[0].targets[n].properties.bucket	Solo para load_type=S3. Ruta destino de archivos.	s3://aw-dev-s3-sdl-datahub-process-058528764918-01/tmp/load_s3_dtk4001	NA
process_modules[0].targets[n].properties.name	Solo para load_type=S3. Nombre lógico del dataset de salida.	data_dtk4001	NA
process_modules[0].targets[n].properties.schema	Solo para load_type=athena. Base de datos de Athena/Glue.	d_datalake_perm o d_perm_aws	NA
process_modules[0].targets[n].properties.table	Solo para load_type=athena. Tabla destino.	dtk4001_frmk4	NA

Crear la carpeta SQL (Opcional)

Si el DE configura el archivo.sql, lo debe realizar basado en Spark Sql, y debe subirlo a la sgte ruta de s3:

s3://aw-dev-s3-sdl-datahub-process-058528764918-01/process/{process_id} /sql/query_DTK4001.sql significa:

- bucket: aw-dev-s3-sdl-datahub-process-058528764918-01
- carpeta lógica del proceso: process/{process_id} /sql/
- archivo SQL: query_{process_id}.sql

En tu flujo FRMK4, ese archivo es el que se usa cuando el transform_sql del proceso apunta a query_DTK4001.sql (por ejemplo en transform_id = t1) para ejecutar la transformación en el job de EMR.

Cada process_id define un patrón de salida múltiple:

1. Genera un output en S3 (perfil_transicion_pqr, formato parquet).
2. Expone/publica el mismo resultado en Athena en d_datalake_perm.TABLE_NAME_FRMK4.
3. Repite publicación en Athena en d_perm_aws.TABLE_NAME_FRMK4.

Cada ítem de la tabla aw-dev-ddb-sdl-datahub-config-process-01 se usa como **configuración de enrutamiento de salidas** dentro del pipeline de datos. Su función es indicar, en tiempo de ejecución, **a qué destinos debe publicarse el resultado del proceso** y con qué formato.

En la práctica se usa cuando el flujo (por ejemplo Step Functions/Lambda/EMR) termina una transformación y necesita resolver dinámicamente:

- salida en S3 (ruta y nombre lógico del dataset), y
- publicación en Athena (tabla y esquema destino, incluso en más de un esquema).

Config process agrupado

Un proceso agrupado permite procesar **varios archivos fuente** dentro de un mismo process_id, en una sola ejecución y con una misma fecinformacion.

```
{
  "user_developer_id": "S38371",
  "user_technical_leader_id": "S38371",
  "user_team": "campañas_contextuales",
  "process_id": "DTK005",
  "batch_group_files_ind": "y",
  "process_type": "CM",
  "process_modules": [
    {
      "extract": [
        {
          "extract_type": "file",
          "properties": {
            "format": "text",
            "path": "/data/desa/landingzone/hadoop/interfacesUser/input",
            "columns": "input_columns.txt",
            "batch_group_files": {
                                "require_all_files" : "n",
                                "files" :[
                                        {"source_name": "ADV_NBGSRQ", "extension": "txt", "delimiter": "|^" ,"header": "n" }
                                        ,{"source_name": "BGA_NBGSRQ", "extension": "txt", "delimiter": "~^" ,"header": "n" }
                                        ,{"source_name": "NGAV_JSASA", "extension": "csv", "delimiter": ";" ,"header": "n" }
                                ]
                            },
            "clean": {
              "type": "shell_script",
              "process": "limpia_CNX_CR_AUT_DEN_PLIN.sh",
              "enabled_ind": "Y"
            },
            "reject": {
              "threshold_percent": "15",
              "enabled_ind": "N"
            },
            "data_protection": {
              "name_method": "ENCRYPTION_AES_CTR",
              "columns_sensitive": "CIF_KEY",
              "enabled_ind": "N"
            }
          }
        }
      ]
    }
  ]
}
```

DTK005.json

En agrupados, fecinformacion debe venir informada en la ejecución. Si no se informa, el framework lanza error: “debe presentar fecinformacion un proceso agrupado”.

- Si **batch_group_files_ind = "y"**, el framework toma **batch_group_files.files[]** como lista de entradas.
- Cada archivo del grupo se **busca y procesa individualmente** (clean / reject / protección según configuración).
- El resultado de cada archivo se **acumula en un solo DataFrame final** usando **unionByName**.
- Si es agrupado, se **añade la columna ID_FILE** (si no existe) con el **source_name** de origen.
- Si **un archivo falla**:
    - **require_all_files = "y"** → se **corta el proceso**.

- **require_all_files = "n"** → el proceso **continúa** y **registra el error** de ese archivo.

- Si **todos los archivos fallan**, el **proceso termina en error**.

Ahora, estos son los comandos con sus parámetros correspondientes para ejecutar cada tipo de proceso.

Tipo	Cómo se ejecuta	¿Requiere fecinformacion en el comando?	Ejemplo sh
CM individual (process_type: "CM" y sin batch_group_files_ind: "y")	Un solo process_id.	No obligatoria (opcional).	Con fecha: sh /data/desa/IBKProjects/DataHubFrmk4/framework/execute_frmk4.sh "DTK001" "20260210" / Sin fecha: sh /data/desa/IBKProjects/DataHubFrmk4/framework/execute_frmk4.sh "DTK001"
CM individual (process_type: "CM" y sin batch_group_files_ind: "y")	Un solo process_id.	No obligatoria (opcional).	Con fecha: sh /data/desa/IBKProjects/DataHubFrmk4/framework/execute_frmk4.sh "DTK001" "20260210" / Sin fecha: sh /data/desa/IBKProjects/DataHubFrmk4/framework/execute_frmk4.sh "DTK001"
CM individual (process_type: "CM" y sin batch_group_files_ind: "y")	Un solo process_id.	No obligatoria (opcional).	Con fecha: sh /data/desa/IBKProjects/DataHubFrmk4/framework/execute_frmk4.sh "DTK001" "20260210" / Sin fecha: sh /data/desa/IBKProjects/DataHubFrmk4/framework/execute_frmk4.sh "DTK001"

El parámetro fecinformacion presenta estas configuraciones:

- fecinformacion **obligatoria** solo en procesos con batch_group_files_ind: "y".
- En CM normal, fecinformacion es opcional.
- En CC cíclico el parámetro principal es el **grupo**, no fecinformacion.

1.2 Crear la metadata del proceso en ruta del server

La carpeta metadata, se debe crear en cada proceso para poder listar los nombres de las columnas, debe ser configurado en un archivo .txt el cual debe tener el nombre de input_columns.txt. Este nombre de archivo debe ser configurado en el config_process del proceso como "columns": "input_columns.txt"

1.3 Crear Shell script en ruta del server (Opcional)

La Shell de limpieza se debe configurar en la carpeta del proceso con el nombre “Shell”, dentro de la subcarpeta, debes crear el archivo Shell .sh, el cual puede ser de dos tipos, Shell_script y Linux_cmd.

Cierre de Documento

Este manual establece las pautas necesarias para la correcta configuración, implementación y seguimiento de los procesos a cargo del Data Engineer, proporcionando una guía clara y estructurada para asegurar la estandarización operativa, la calidad de los datos y la continuidad de los flujos de procesamiento.

La adecuada aplicación de los lineamientos aquí descritos permitirá reducir errores en la configuración, optimizar tiempos de despliegue y facilitar las actividades de mantenimiento, monitoreo y soporte de los distintos componentes involucrados en la arquitectura de datos. Asimismo, este documento busca servir como referencia técnica para garantizar que cada proceso sea implementado bajo criterios de orden, trazabilidad, seguridad y eficiencia.

Finalmente, se recomienda que este manual sea utilizado como documento base de consulta y que sea actualizado de manera periódica, en función de los cambios en la infraestructura, las herramientas tecnológicas y los requerimientos del negocio, asegurando así su vigencia y utilidad dentro del ciclo de gestión de datos.